

Arquitectura de Microservicios con Inteligencia Artificial para la Gestión Inteligente de Incidentes Urbanos

Manuel Krivoy

Resumen—Este trabajo presenta el diseño de una arquitectura basada en microservicios para la gestión de incidentes urbanos, incorporando técnicas de inteligencia artificial para mejorar la clasificación, priorización y derivación de reclamos ciudadanos.

La problemática actual se centra en sistemas centralizados que presentan limitaciones en escalabilidad, trazabilidad y eficiencia operativa, especialmente ante altos volúmenes de incidentes.

Como solución, se propone una arquitectura desacoplada junto con un módulo de procesamiento de lenguaje natural (NLP) capaz de interpretar descripciones textuales y automatizar decisiones clave. El trabajo busca demostrar la viabilidad técnica de este enfoque en un contexto controlado.

I. INTRODUCCIÓN

En entornos urbanos modernos, la gestión eficiente de incidentes representa un desafío creciente debido al aumento en la cantidad de reportes ciudadanos y la complejidad operativa de su resolución.

Los sistemas tradicionales, generalmente monolíticos, presentan dificultades para escalar, mantener trazabilidad y responder de forma eficiente. Además, la dependencia de procesos manuales para clasificar y derivar incidentes introduce errores, demoras y falta de consistencia.

En este contexto, surge la necesidad de soluciones tecnológicas que permitan automatizar y optimizar estos procesos, garantizando mayor eficiencia, escalabilidad y calidad de servicio.

El objetivo de este trabajo es proponer una arquitectura basada en microservicios e inteligencia artificial para mejorar la clasificación, priorización, asignación y seguimiento de incidentes urbanos.

II. CASO DE ESTUDIO

El caso de estudio se centra en la gestión de incidentes urbanos reportados por ciudadanos mediante una plataforma digital. Estos incidentes incluyen, entre otros:

- Fallas en alumbrado público.
- Problemas en recolección de residuos.
- Baches o deterioro de infraestructura vial.
- Situaciones de seguridad urbana.

La gestión de estos incidentes requiere:

- Recepción y registro de reclamos.
- Interpretación de descripciones textuales.
- Clasificación por tipo de incidente.
- Determinación de prioridad.
- Asignación a áreas responsables.

- Seguimiento del estado del incidente.

Actualmente, estos procesos suelen realizarse de forma manual o semi-automatizada, generando ineficiencias operativas, errores en la clasificación y demoras en la resolución.

El alcance del trabajo se limita al diseño y validación técnica de una solución que aborde estas problemáticas, sin contemplar su implementación en un entorno productivo real.

II-A. Aplicación del método ATAM

Se utiliza el método ATAM (Architecture Tradeoff Analysis Method) para identificar los atributos de calidad relevantes y guiar las decisiones arquitectónicas [1].

En el contexto de este trabajo, ATAM permite relacionar la problemática del caso de estudio con los drivers arquitectónicos que condicionan el diseño de la solución. De este modo, la elección de una arquitectura basada en microservicios no se presenta como una decisión únicamente tecnológica, sino como una respuesta a necesidades concretas de calidad del sistema.

II-B. Drivers arquitectónicos

A partir del análisis del caso de estudio, se identifican los siguientes drivers arquitectónicos principales:

- **Escalabilidad:** capacidad de soportar altos volúmenes de incidentes concurrentes.
- **Disponibilidad:** necesidad de mantener el sistema operativo ante fallas parciales.
- **Modificabilidad:** posibilidad de incorporar cambios o nuevos componentes sin afectar toda la solución.
- **Desempeño:** capacidad de responder de forma eficiente ante operaciones críticas, como la clasificación o consulta de incidentes.
- **Interoperabilidad:** capacidad de integrarse con sistemas municipales, aplicaciones externas o múltiples clientes.

II-C. Escenarios de calidad

Los drivers anteriores se expresan en los siguientes escenarios de calidad:

- Ante un aumento masivo de incidentes, por ejemplo durante una tormenta o evento crítico, el sistema debe poder escalar los componentes más demandados.
- Ante la necesidad de incorporar un nuevo modelo de inteligencia artificial, el sistema debe permitir modificar

el servicio de clasificación sin afectar el resto de los servicios.

- Ante la integración con sistemas externos municipales, la solución debe exponer interfaces claras y reutilizables.
- Ante la falla de un componente, el sistema debe evitar que todo el flujo de gestión de incidentes quede indisponible.

III. MARCO TEÓRICO

III-A. Arquitectura de microservicios

La arquitectura de microservicios propone la construcción de sistemas como un conjunto de servicios pequeños, independientes y desplegables de manera autónoma, que se comunican mediante APIs [2]. Este enfoque permite mejorar la escalabilidad, mantenibilidad y resiliencia.

En este tipo de arquitectura, cada servicio encapsula una responsabilidad específica del dominio, lo que favorece el bajo acoplamiento y la alta cohesión. Esto permite que los componentes evolucionen de forma independiente, aunque también introduce nuevos desafíos asociados a la comunicación entre servicios, monitoreo, despliegue y administración distribuida.

III-B. Procesamiento de lenguaje natural

El procesamiento de lenguaje natural (NLP) permite analizar texto no estructurado para extraer información relevante, siendo ampliamente utilizado en tareas de clasificación automática [3].

En el contexto de la gestión de incidentes urbanos, el NLP puede utilizarse para interpretar la descripción textual ingresada por el ciudadano y colaborar en la identificación del tipo de incidente, su prioridad y el área responsable de resolución.

III-C. Método ATAM

El método ATAM (Architecture Tradeoff Analysis Method) es una técnica utilizada para evaluar decisiones arquitectónicas en función de atributos de calidad, escenarios relevantes y compromisos entre distintas alternativas de diseño [1].

Su aplicación permite identificar qué decisiones favorecen determinados atributos, como escalabilidad o disponibilidad, y qué costos o riesgos pueden introducir. Por este motivo, resulta útil para justificar la elección de una arquitectura en función de los requerimientos reales del sistema y no únicamente por preferencias tecnológicas.

IV. DESARROLLO

IV-A. Introducción al desarrollo

A partir de la problemática identificada, se plantea la necesidad de una solución que permita desacoplar responsabilidades, escalar de forma independiente y automatizar la toma de decisiones sobre los incidentes.

En este sentido, se opta por una arquitectura de microservicios complementada con un componente de inteligencia artificial. Esta decisión se fundamenta en los drivers arquitectónicos identificados mediante ATAM, especialmente la escalabilidad, modificabilidad, disponibilidad e interoperabilidad.

IV-B. Requerimientos del sistema

Los requerimientos del sistema se derivan de los drivers arquitectónicos identificados mediante ATAM y de las limitaciones observadas en los sistemas tradicionales de gestión de incidentes.

A partir de dicho análisis, se identifican los siguientes requerimientos principales:

- **Escalabilidad:** el sistema debe soportar un alto volumen de incidentes concurrentes sin degradar su rendimiento.
- **Desacoplamiento:** las distintas funcionalidades deben poder evolucionar de forma independiente.
- **Automatización:** se requiere minimizar la intervención manual en la clasificación y asignación de incidentes.
- **Trazabilidad:** el sistema debe permitir el seguimiento completo del ciclo de vida de cada incidente.
- **Flexibilidad:** debe ser posible incorporar mejoras, como nuevos modelos de inteligencia artificial, sin afectar el resto del sistema.
- **Interoperabilidad:** la solución debe permitir la integración con aplicaciones web, móviles, chatbots o sistemas municipales externos.

Estos requerimientos surgen directamente de las limitaciones observadas en los sistemas tradicionales, particularmente en términos de rigidez estructural, dependencia de procesos manuales y dificultad para escalar.

IV-C. Arquitectura propuesta

En función de los requerimientos definidos y de los drivers arquitectónicos identificados mediante ATAM, se propone una arquitectura basada en microservicios, orientada a desacoplar responsabilidades, facilitar la escalabilidad y permitir la evolución independiente de los distintos componentes del sistema.

La solución se plantea como una plataforma digital para la gestión inteligente de incidentes urbanos, donde los ciudadanos pueden registrar reclamos mediante una interfaz web o canales alternativos, como una integración con chatbot. A partir de la descripción textual del incidente, un componente de procesamiento de lenguaje natural permite clasificar el reclamo, estimar su prioridad y derivarlo al área correspondiente.

La arquitectura propuesta contempla dos niveles de implementación: una arquitectura ideal, pensada para un entorno productivo y escalable, y una arquitectura reducida, orientada a la prueba de concepto.

IV-C1. Arquitectura ideal: En un escenario productivo, la arquitectura se plantea como una solución desplegada en la nube, utilizando servicios contenedorizados y administrados sobre una infraestructura escalable. Una alternativa posible es utilizar AWS como proveedor cloud, desplegando los microservicios en contenedores Docker orquestados mediante Kubernetes.

Esta infraestructura permitiría ejecutar cada microservicio de forma independiente, escalar aquellos componentes con mayor demanda y mejorar la disponibilidad general del sistema. En este esquema, el tráfico externo ingresaría a través de un balanceador de carga o API Gateway, que centralizaría el acceso a los servicios expuestos.

La arquitectura ideal contempla los siguientes componentes:

- **Cliente web:** interfaz utilizada por ciudadanos u operadores para registrar y consultar incidentes.
- **Chatbot o canal conversacional:** canal adicional para registrar incidentes mediante lenguaje natural.
- **API Gateway / Load Balancer:** punto de entrada centralizado para las solicitudes externas, encargado de enrutar las peticiones hacia los microservicios correspondientes.
- **Servicio de recepción:** responsable de registrar incidentes, validar datos de entrada y persistir la información inicial.
- **Servicio de clasificación:** encargado de procesar la descripción textual del incidente mediante técnicas de procesamiento de lenguaje natural, determinando categoría y prioridad.
- **Servicio de asignación:** responsable de aplicar reglas de negocio para derivar el incidente al área municipal correspondiente.
- **Servicio de seguimiento:** encargado de consultar y actualizar el estado del incidente a lo largo de su ciclo de vida.
- **Servicio de autenticación:** componente encargado de identificar a cada municipio o cliente que consume la plataforma.
- **Base de datos:** repositorio persistente para almacenar incidentes, estados, áreas responsables, usuarios y trazabilidad.
- **Monitoreo y logs:** mecanismos para observar el comportamiento de los servicios, detectar fallas y analizar métricas de uso.

Desde el punto de vista de infraestructura, cada microservicio podría ejecutarse en un contenedor independiente. Kubernetes permitiría administrar el ciclo de vida de estos contenedores, realizar escalado horizontal, reiniciar servicios ante fallas y distribuir la carga de trabajo. Asimismo, un balanceador de carga permitiría distribuir las solicitudes entrantes, mejorando la disponibilidad y el desempeño del sistema.

Este enfoque resulta adecuado para un modelo Software as a Service (SaaS), ya que permite que múltiples municipios consuman la misma plataforma de forma aislada mediante mecanismos de autenticación y control de acceso. Por ejemplo, cada municipio podría identificarse mediante credenciales específicas, como `client_id` y `client_secret`, permitiendo diferenciar su consumo y administrar sus incidentes de forma independiente.

De esta manera, la arquitectura ideal responde a los drivers de escalabilidad, disponibilidad, interoperabilidad y modificabilidad, permitiendo que la solución pueda crecer progresivamente y adaptarse a distintos contextos municipales.

IV-C2. Arquitectura reducida para la prueba de concepto: Si bien la arquitectura ideal contempla una infraestructura cloud escalable, la prueba de concepto implementa una versión reducida y simplificada, cuyo objetivo principal es validar la viabilidad técnica del flujo de gestión de incidentes sin incurrir en costos de infraestructura productiva.

La POC no busca demostrar escalabilidad real en producción ni desplegar una infraestructura cloud completa, sino validar el comportamiento funcional de la solución propuesta.

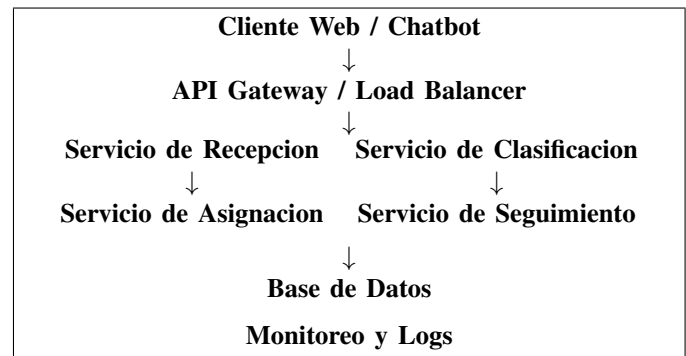


Figura 1. Arquitectura ideal propuesta para un entorno productivo escalable.

Por este motivo, se prioriza una implementación local o de bajo costo, manteniendo la separación lógica de componentes.

La arquitectura reducida contempla los siguientes elementos:

- **Frontend web:** aplicación simple con un formulario para registrar incidentes y consultar su estado.
- **Backend principal:** servicio encargado de recibir los incidentes y coordinar el flujo de clasificación y asignación.
- **Módulo de clasificación:** componente simplificado que analiza la descripción del incidente utilizando reglas básicas, palabras clave o un modelo liviano de NLP.
- **Módulo de asignación:** lógica encargada de derivar el incidente a un área simulada según la categoría detectada.
- **Base de datos local:** almacenamiento de incidentes, categorías, prioridades y estados.

En esta versión, los microservicios pueden representarse como módulos separados dentro de un mismo backend o como servicios livianos ejecutados localmente mediante Docker Compose. Esta decisión permite mantener el concepto de separación de responsabilidades sin requerir una infraestructura compleja como Kubernetes o servicios cloud pagos.

La prueba de concepto se enfoca en validar el siguiente flujo:

- Registro de un incidente mediante una interfaz web.
- Envío de una descripción textual del problema.
- Clasificación automática del incidente.
- Determinación de prioridad.
- Asignación a un área responsable simulada.
- Consulta del estado del incidente.

De este modo, la POC permite demostrar el valor agregado de la solución sin necesidad de implementar todos los componentes productivos. La infraestructura reducida conserva los principios principales de la arquitectura propuesta, especialmente el desacoplamiento, la automatización y la trazabilidad, pero evita costos y complejidad innecesaria para esta etapa del trabajo.

IV-C3. Relación entre arquitectura ideal y POC: La arquitectura reducida de la POC representa una validación parcial de la arquitectura ideal. Mientras que la arquitectura productiva contempla despliegue cloud, contenedores, balanceadores, monitoreo y escalabilidad horizontal, la POC se concentra en validar el flujo funcional principal.

Esta diferencia permite separar claramente dos objetivos:

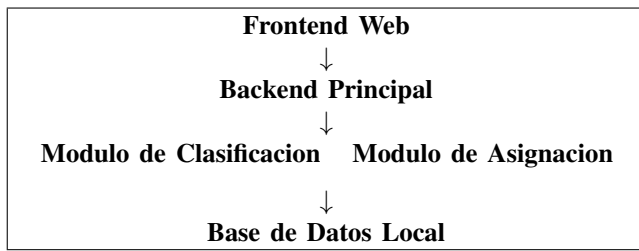


Figura 2. Arquitectura reducida de la prueba de concepto.

- **Arquitectura ideal:** demostrar cómo debería diseñarse la solución para un entorno real, escalable y multi-municipio.
- **POC:** demostrar que el flujo de registro, clasificación, asignación y seguimiento es técnicamente viable.

Por lo tanto, la POC no reemplaza la arquitectura ideal, sino que funciona como una primera validación técnica de sus componentes principales. En futuras etapas, los módulos validados podrían evolucionar hacia microservicios independientes desplegados en una infraestructura cloud productiva.

IV-D. Análisis de trade-offs arquitectónicos

La aplicación de ATAM permite identificar no solo los beneficios de la arquitectura propuesta, sino también los compromisos asociados a la elección de microservicios.

Uno de los principales beneficios es la escalabilidad independiente de los componentes. Sin embargo, este atributo implica un aumento en la complejidad operativa, ya que cada servicio requiere mecanismos propios de despliegue, monitoreo, comunicación y mantenimiento.

Otro compromiso relevante se observa entre el desacoplamiento y la latencia. La separación de responsabilidades permite que cada servicio evolucione de forma autónoma, pero al mismo tiempo introduce comunicaciones remotas entre componentes, lo cual puede afectar el desempeño si no se diseñan adecuadamente las interfaces.

También existe un compromiso entre flexibilidad y complejidad de integración. La exposición de APIs facilita la integración con múltiples municipios, aplicaciones externas o interfaces conversacionales, pero requiere definir mecanismos de autenticación, autorización, versionado y control de acceso.

En relación con el componente de inteligencia artificial, separar el servicio de clasificación permite mejorar o reemplazar el modelo de NLP sin modificar el resto del sistema. No obstante, esto exige considerar la disponibilidad de dicho servicio, ya que su falla podría afectar la clasificación automática de incidentes.

En síntesis, la arquitectura de microservicios permite satisfacer los principales drivers identificados, aunque introduce costos adicionales en términos de complejidad técnica, administración distribuida y diseño de comunicaciones. Estos compromisos son aceptables dentro del alcance del trabajo, dado que la prioridad del sistema se centra en la escalabilidad, modificabilidad e interoperabilidad.

IV-E. Interfaces entre servicios y capa de presentación

La interacción entre servicios se realiza mediante endpoints REST, donde cada microservicio expone operaciones específicas para el intercambio de información. Entre las principales interfaces se contemplan:

- Registro de incidente.
- Consulta de clasificación.
- Asignación a área.
- Consulta de estado.

Además de las interfaces entre servicios, la solución contempla una capa de presentación orientada al usuario final. Esta interfaz puede implementarse mediante una aplicación web sencilla, cuya función principal sea permitir la carga y consulta de incidentes. En una primera instancia, no se requiere una interfaz compleja, sino una solución funcional que incluya:

- Un formulario para registrar incidentes.
- Un campo de descripción textual del problema.
- Visualización de información relevante del incidente.
- Consulta del estado o seguimiento del reclamo.

Adicionalmente, la arquitectura permite la integración con interfaces conversacionales, como chatbots, que utilicen las mismas APIs expuestas por los microservicios. Esto habilita nuevos canales de interacción con el usuario, donde el ciudadano puede reportar incidentes mediante lenguaje natural, mejorando la accesibilidad y la experiencia de uso.

De este modo, la arquitectura no solo contempla la comunicación entre componentes internos, sino también la interacción con el usuario a través de distintos canales, manteniendo un diseño flexible y extensible.

Por otro lado, la exposición de funcionalidades mediante APIs permite que la solución sea consumida por distintos clientes, como aplicaciones web, móviles o sistemas externos, posicionando a la arquitectura como una plataforma extensible e integrable.

Este enfoque refuerza el modelo de reutilización del sistema en distintos municipios o entidades, facilitando su adopción sin necesidad de desarrollos específicos por cada caso.

IV-F. Prueba de concepto

Se plantea una prueba de concepto orientada a validar la viabilidad técnica de la solución.

La misma contempla:

- Registro de incidentes con descripción textual.
- Procesamiento del texto mediante un modelo básico de NLP.
- Clasificación automática del incidente.
- Asignación a un área simulada.

El modelo de NLP podrá basarse en técnicas simples como análisis de palabras clave o modelos preentrenados, evitando complejidad innecesaria para esta etapa.

IV-G. Análisis y discusión

La arquitectura propuesta permite abordar las principales problemáticas detectadas, especialmente en términos de escalabilidad, modificabilidad y automatización.

El uso de inteligencia artificial reduce la dependencia de procesos manuales, mejorando la eficiencia y consistencia en la gestión de incidentes.

Desde el análisis realizado mediante ATAM, la solución se encuentra alineada con los drivers arquitectónicos definidos. En particular, la separación de servicios permite escalar componentes críticos, modificar el módulo de clasificación sin afectar el resto del sistema e integrar la plataforma con distintos canales o municipios.

No obstante, la arquitectura también introduce desafíos vinculados a la complejidad operativa, la comunicación entre servicios y la necesidad de mecanismos de monitoreo, autenticación y control de fallos. Estos aspectos deberán considerarse en futuras etapas si la solución se implementa en un entorno productivo.

Si bien la solución propuesta no contempla su implementación en un entorno productivo, los resultados esperados de la prueba de concepto permiten validar su aplicabilidad en escenarios reales.

Asimismo, el enfoque orientado a servicios y la posibilidad de consumo mediante APIs habilita escenarios de adopción multi-organización, alineándose con modelos modernos de plataformas SaaS.

V. CONCLUSIÓN

El trabajo presenta una solución basada en microservicios e inteligencia artificial para mejorar la gestión de incidentes urbanos.

La arquitectura propuesta permite desacoplar responsabilidades, escalar de forma eficiente y automatizar procesos clave como la clasificación y asignación de incidentes.

A partir de la aplicación del método ATAM, se identificaron drivers arquitectónicos como escalabilidad, disponibilidad, modificabilidad, desempeño e interoperabilidad, los cuales permitieron justificar la elección de una arquitectura basada en microservicios.

El análisis de trade-offs muestra que esta elección aporta beneficios relevantes para el caso de estudio, aunque también introduce desafíos técnicos asociados a la complejidad operativa y la comunicación entre servicios.

Los resultados esperados demuestran la viabilidad técnica del enfoque y su potencial impacto en la optimización de sistemas de gestión urbana.

VI. LÍNEAS FUTURAS DE INVESTIGACIÓN

Como trabajos futuros se propone:

- Implementación en entornos reales.
- Mejora del modelo de inteligencia artificial.
- Integración con sistemas municipales existentes.
- Incorporación de analítica avanzada.
- Evaluación de mecanismos de monitoreo, resiliencia y observabilidad para servicios distribuidos.

REFERENCIAS

- [1] R. Kazman, M. Klein, and P. Clements, "The atam: Method for architecture evaluation," in *Proceedings of the 19th International Conference on Software Engineering*. IEEE, 1998, pp. 68–78.
- [2] S. Newman, *Building Microservices*. O'Reilly Media, 2015.
- [3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. Prentice Hall, 2023.